

Жизненный цикл ПО

Планирование и анализ требований

Определение требований

Проектирование архитектуры

Разработка

Тестирование

Развертывание

Эксплуатация и сопровождение

Вывод из эксплуатации

Планирование и анализ требований

Что происходит на этапе:

- Собираются бизнес-требования к ПО, т.е описание задач, которые должен решать продукт;
- Опрос стейкхолдеров;
- Выявление аудитории, которая будет пользоваться решением;
- Оценка ресурсов, которые нужны для реализации: количество сотрудников, время, бюджет;
- Согласование критериев, по которым будут оценивать успешность решения.

Участники: руководитель проекта, владелец продукта, заказчик, бизнес-аналитик, стейкхолдеры.

Определение требований

Что происходит на этапе:

- Уточняются все требования к целевому программному обеспечению.
- Требования получают одобрение со стороны клиентов, аналитиков рынка и всех заинтересованных сторон.
- Создаются артефакты в виде документации, в которых указаны функциональные и нефункциональные требования.

Участники: руководитель проекта, владелец продукта, команда разработки, архитектор.

Проектирование архитектуры

Что происходит на этапе:

- Создаются верхнеуровневая и нижнеуровневая архитектура системы
- Результатом этапа будет проектная документация с информацией, которая необходима для реализации решения - спецификация.

Участники: архитекторы, UX/UI-дизайнеры, QA-инженеры, команда разработки

Архитектура

Верхнеуровневая	Низкоуровневая
Основные компоненты системы — например, модули, сервисы или подсистемы	Принцип работы отдельных компонентов системы
Принцип взаимодействия этих компонентов между собой	Алгоритмы, структуры данных и логика работы внутри каждого модуля
Общая архитектура системы — например, клиент-серверная архитектура или микросервисы	Детали реализации — например, какие классы, методы или функции будут созданы
Зависимость решения от других систем	Взаимодействие с базами данных, API или другими внешними системами
Прототип интерфейса	

Разработка

Что происходит на этапе:

- Разработчики пишут код, интегрируют компоненты и проводят модульное тестирование.
- Итогом становится рабочее решение в соответствии с техническими спецификациями.

Участники: разработчики, техлиды, DevOps-инженеры, QA-инженеры

Тестирование

Что происходит на этапе:

- Проверка ПО на ошибки и соответствие спецификации.
- Проверка включает интеграционное, системное и пользовательское тестирование.

Участники: QA-инженеры, разработчики, автоматизаторы тестирования, потенциальные пользователи в случае User Acceptance Testing.

Развертывание

Что происходит на этапе:

- Программу передают заказчику и выпускают в «боевую среду» - устанавливают на серверы или выкладывают в магазины приложений, чтобы аудитория могла им пользоваться.
- Дополнительно готовят пользовательскую документацию.
- Обучают пользователей

Участники: DevOps-инженер, разработчики, руководитель проекта.

Эксплуатация и сопровождение

Что происходит на этапе:

- ПО обновляют под запросы пользователей
- устраняют ошибки при появлении
- оказывают техническую поддержку пользователям.

Участники: техническая поддержка

Вывод из эксплуатации

Что происходит на этапе:

- Команда прекращает обслуживать продукт или заменяет его новой версией, потому что ПО устарело или такая инициатива поступила от заказчика

Участники: техническая поддержка

Функциональные требования

Определяют все функции, которые программа или продукт должны выполнять в зависимости от того, какие действия совершает пользователь.

Отвечают на вопрос **что должна делать система?**

Нефункциональные требования

Нефункциональные требования описывают то, как система должна работать.

Некоторые НФТ:

- Надежность (Reliability)
- Масштабируемость (Scalability)
- Производительность (Performance)
- Эластичность (Elasticity)
- Возможность развертывания (Deployability)
- Защищенность (Security)
- Эволюционность (Maintainability)
- Простота разработки (Simplicity)

Признаки плохого проекта

- **Закрепощённость**: система с трудом поддается изменениям, поскольку любое минимальное изменение вызывает эффект "снежного кома", затрагивающего другие компоненты системы.
- **Неустойчивость**: в результате осуществляемых изменений система разрушается в тех местах, которые не имеют прямого отношения к непосредственно изменяемому компоненту.
- **Неподвижность**: достаточно трудно разделить систему на компоненты, которые могли бы повторно использоваться в других системах.
- **Вязкость**: сделать что-то правильно намного сложнее, чем выполнить какие-либо некорректные действия.
- **Неоправданная сложность**: проект включает инфраструктуру, применение которой не влечёт непосредственной выгоды.
- **Неопределенность**: исходный код трудно читать и понимать. Недостаточно четко выражено содержимое проекта.

Пример влияния плохой архитектуры

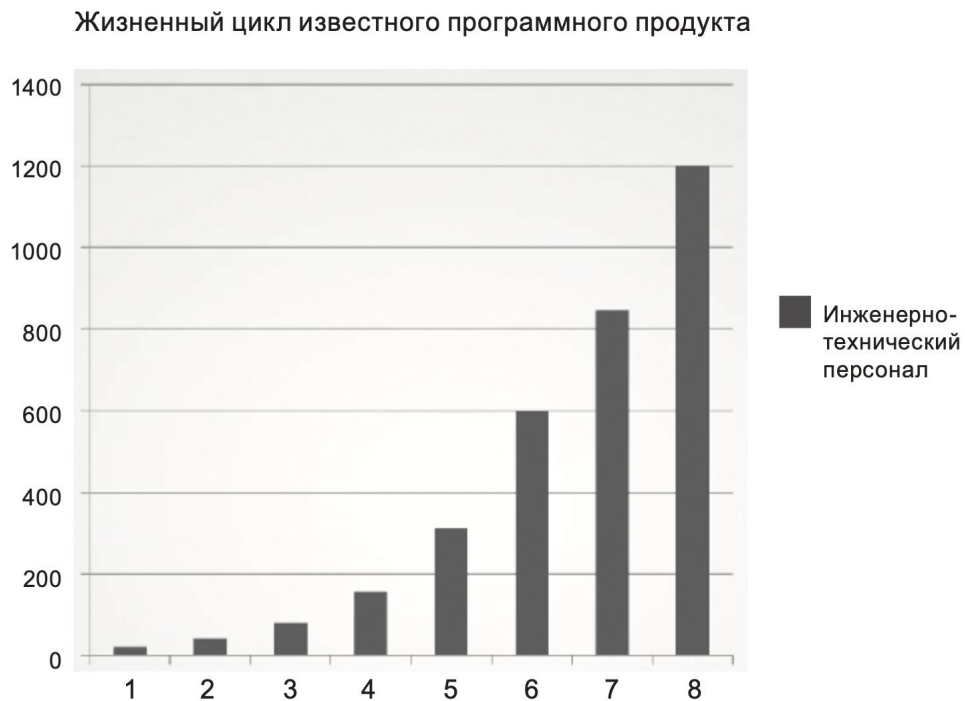


Рис. 1.1. Рост численности инженерно-технического персонала.

Пример влияния плохой архитектуры

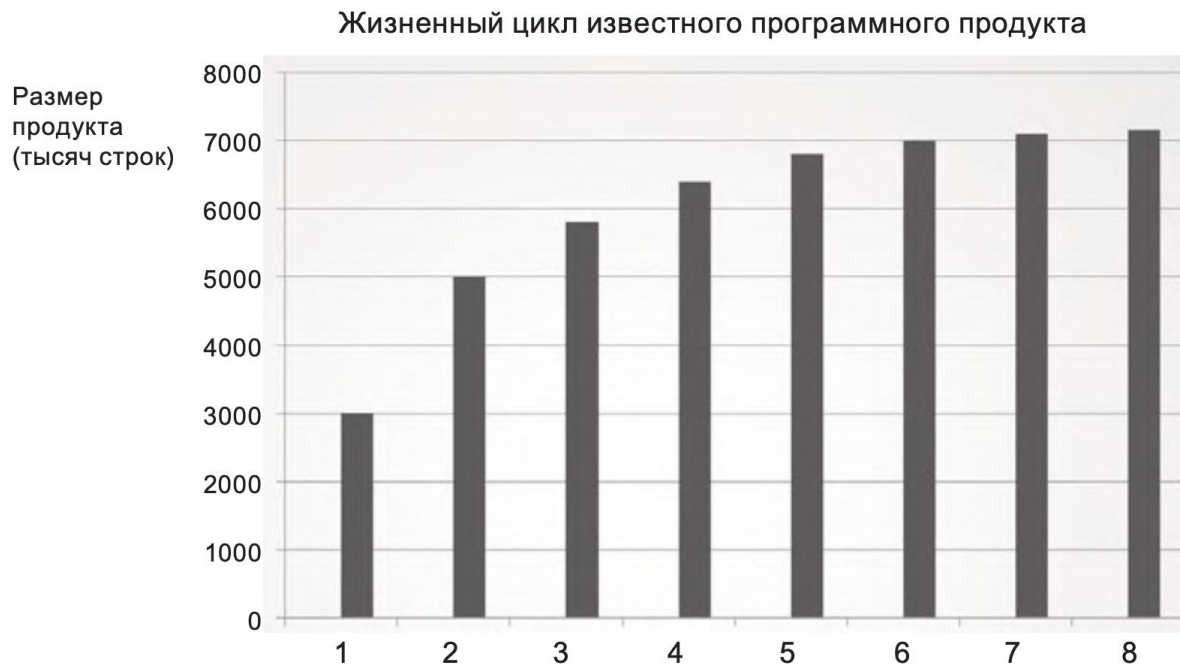


Рис. 1.2. Продуктивность за тот же период

Пример влияния плохой архитектуры

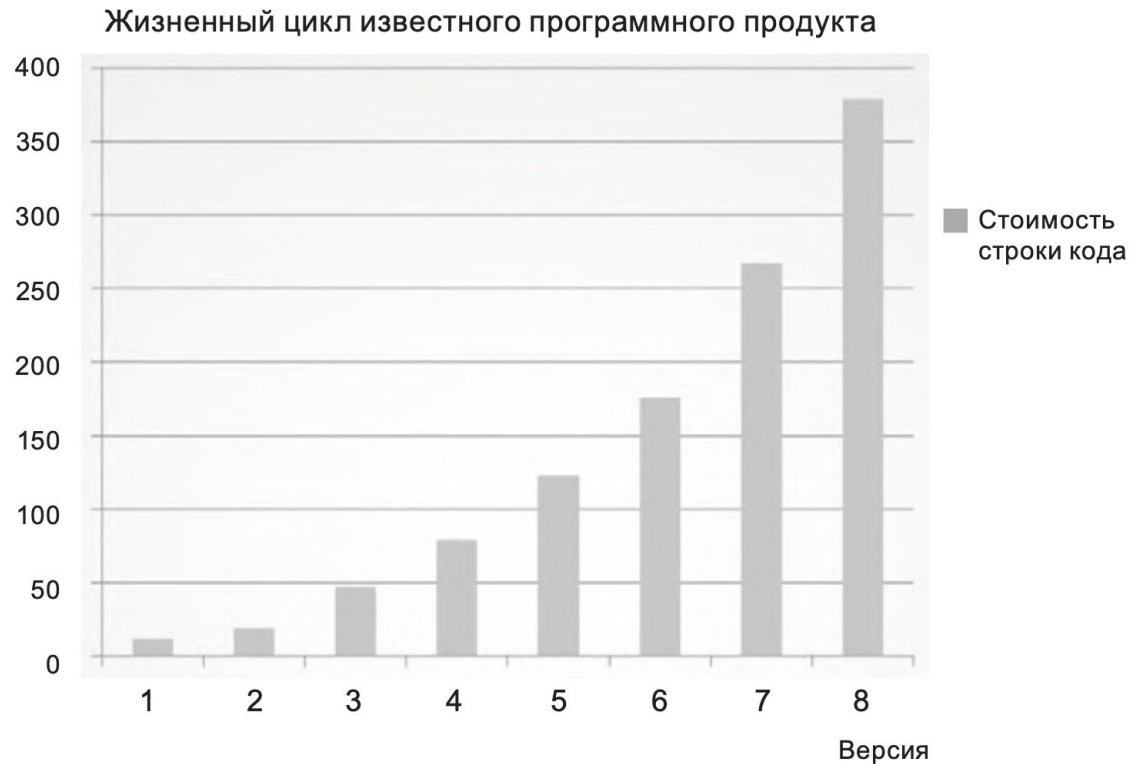


Рис. 1.3. Изменение стоимости строки кода с течением времени

Цитаты

Цель архитектуры ПО - минимизировать количество ресурсов, необходимых для создания и поддержки ПО.

Если программа работает превосходно, но ее нельзя изменить, скоро она станет бесполезной. Если программа не работает, но в нее легко вносятся изменения, после небольших доработок она всегда будет оставаться полезной.

SOLID

- Single Responsibility Principle (Принцип единственной ответственности).
- Open/Closed Principle (Принцип открытости/закрытости).
- Liskov's Substitution Principle (Принцип подстановки Барбары Лисков).
- Interface Segregation Principle (Принцип разделения интерфейса).
- Dependency Inversion Principle (Принцип инверсии зависимостей).

Single Responsibility

У модуля должна быть только одна причина для изменения. Весь код, который меняется по этой причине, должен быть собран в этом модуле.

- помогает разбивать и декомпозировать задачи по одной на модуль;
- уменьшает количество модулей, которые надо изменить при изменении требований;
- ограничивает влияние изменений, помогая контролировать сложность системы.

Open/Closed Principle

Программные сущности должны быть открыты для расширения и закрыты для изменения.

- заставляет проектировать модули так, чтобы они делали только одну вещь и делали её хорошо;
- побуждает связывать сущности через абстракции (а не реализацию) там, где могут поменяться бизнес-требования;
- обращает внимание проектировщиков на места стыка и взаимодействие сущностей;
- позволяет сократить количество кода, который необходимо менять при изменении бизнес-требований;
- делает внесение изменений безопасным и относительно дешёвым.

Принцип подстановки Барбары Лисков

Функции, которые используют базовый тип, должны иметь возможность использовать подтипы базового типа не зная об этом.

- помогает проектировать систему, опираясь на поведение модулей;
- вводит ограничения и правила наследования объектов, чтобы их потомки не противоречили базовому поведению;
- делает поведение модулей последовательным и предсказуемым;
- помогает избегать дублирования, выделять общую для нескольких модулей функциональность в общий интерфейс;
- позволяет выявлять при проектировании проблемные абстракции и скрытые связи между сущностями.

Принцип разделения интерфейсов

- Сущности не должны зависеть от интерфейсов, которые они не используют.
- помогает бороться с наследованием или реализацией ненужной функциональности;
- даёт возможность спроектировать модули так, чтобы их затрагивали изменения только тех интерфейсов, которые они действительно реализуют;
- снижает зацепление модулей;
- уничтожает наследование ради наследования, поощряет использование композиции;
- позволяет выявлять более высокие абстракции и находить неочевидные связи между сущностями.

Принцип инверсии зависимостей

Высокоуровневые модули не должны зависеть от низкоуровневых; оба должны зависеть от абстракций. Абстракции не должны зависеть от деталей. Детали должны зависеть от абстракций.

- вводит правила и ограничения для зависимости одних модулей от других;
- снижает зацепление модулей;
- делает тестирование модулей проще;
- позволяет проектировать систему так, чтобы модули были заменяемы на другие

Модульность

Возможность системы декомпозироваться на несколько внутренне связанных модулей.

В хорошей модульности изменения в одном модуле не затрагивают другие или затрагивают небольшое количество.

Связанность / Coupling

мера того, насколько взаимозависимы разные подпрограммы или модули.

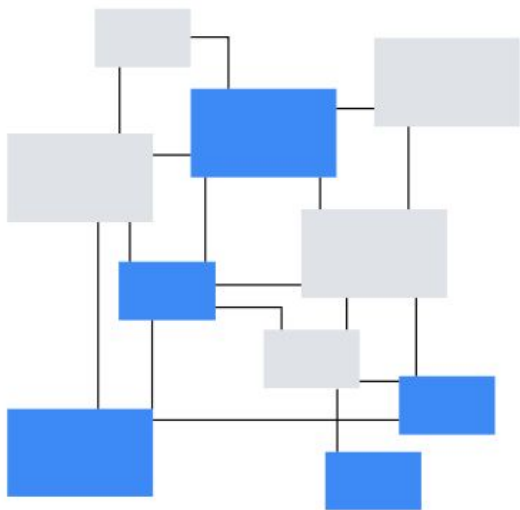
Сильная связанность (high coupling) означает высокую зависимость между модулями.

Слабая связанность (low coupling) - состояние системы, когда изменения в одном модуле не требуют изменений в другом.

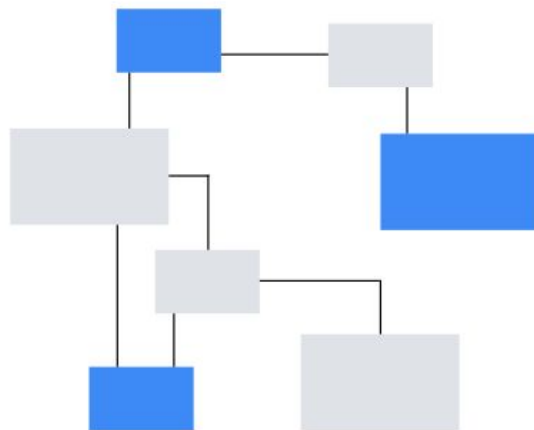
Слабо связанный сервис обладает тремя главными свойствами:

- Малое число зависимостей между подсистемами (модулями)
- Слабая зависимость одного модуля от изменений в другом модуле.
- Легкость повторного использования подсистем.

Связанность / Coupling



High coupling



Low coupling

Связность / Cohesion

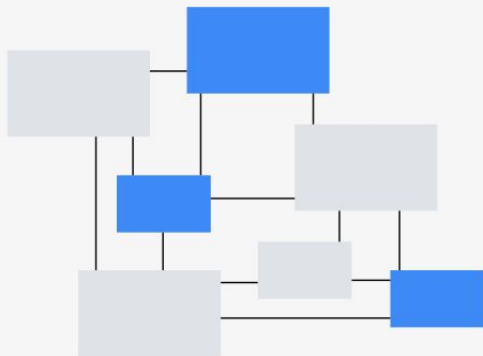
степень взаимосвязанности элементов внутри модуля.

Сильная связность (high cohesion) — состояние, когда обязанности компонентов внутри одного модуля хорошо согласованы между собой и он не выполняет огромных объемов работы.

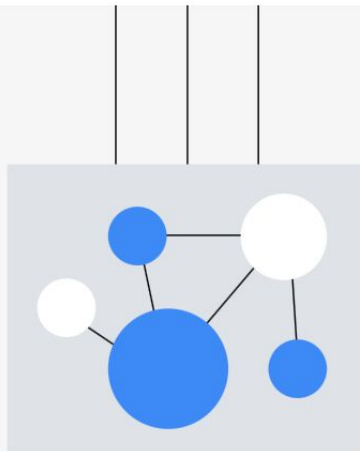
Слабая связность (low cohesion) — состояние, когда один модуль выполняет много разнородных функций. Это свойство модуля приводит к следующим проблемам:

- Трудность понимания
- Сложность при повторном использовании
- Сложность поддержки
- Ненадежность, постоянная подверженность изменениям

Связность / Cohesion

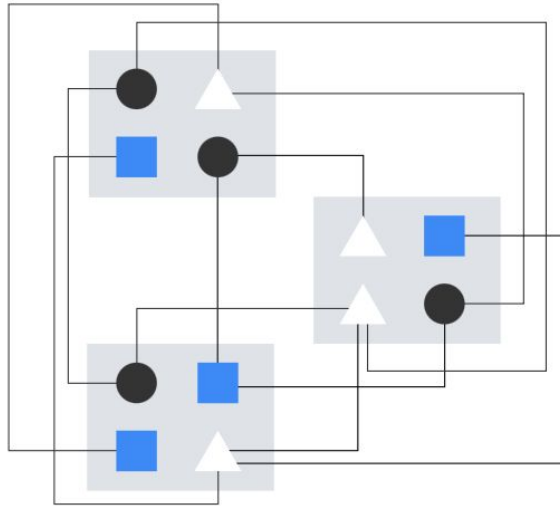


Coupling

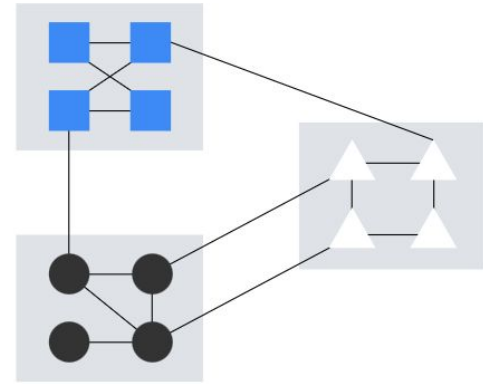


Cohesion

Low coupling - high cohesion



Low Cohesion
High Coupling



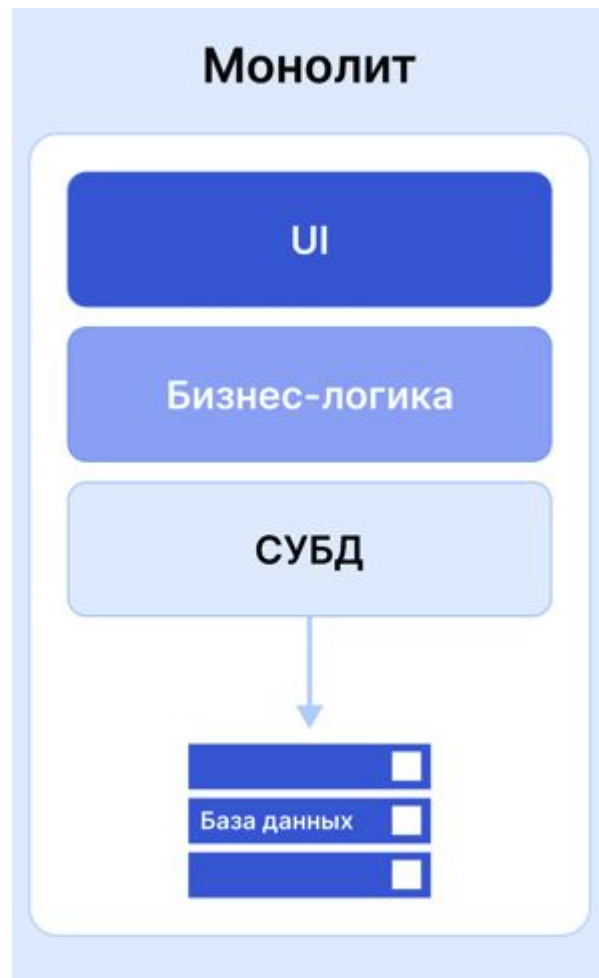
High Cohesion
Low Coupling

Архитектура

Архитектура программного обеспечения — совокупность важнейших решений об организации программной системы.

Монолит

Архитектура, при которой все компоненты системы (логика, интерфейс, база данных) объединены в единую базу и работают как одно целое (единый процесс).

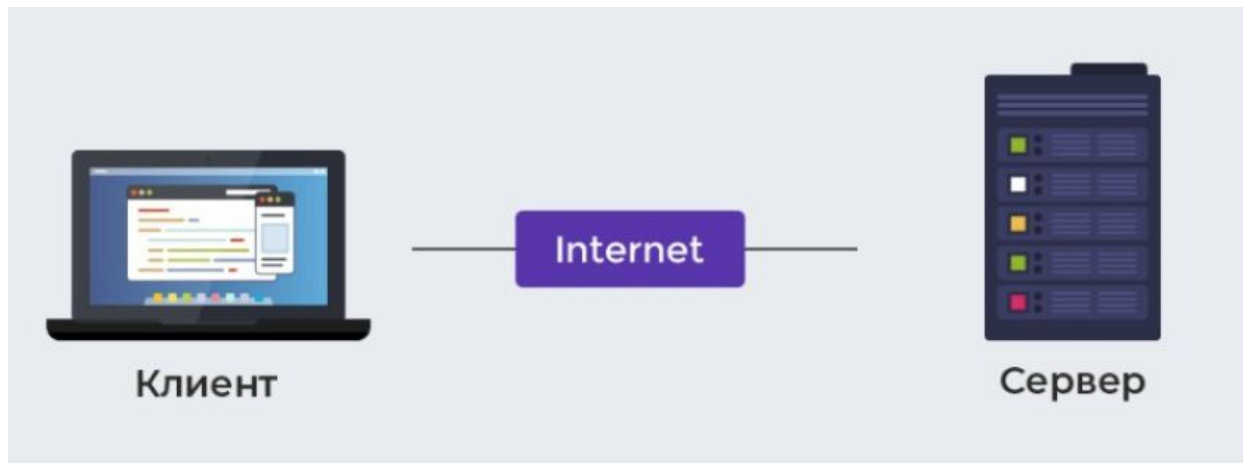


Клиент-серверная архитектура

Логика разделена между клиентом и сервером по сети.

Клиент - браузер, десктоп приложение, мобильное приложение.

Сервер - API + База данных

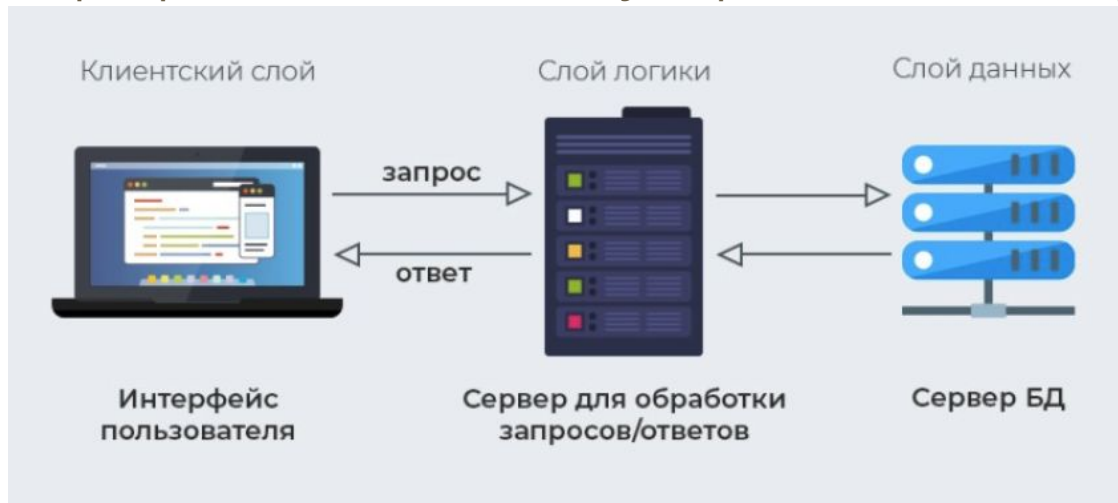


Трехуровневая архитектура

Клиентский слой — интерфейс пользователя.

Слой логики — сервер, на котором происходит обработка запросов/ответов.

Слой данных — сервер баз данных: к нему обращается наш сервер.



Микросервисная

Микросервисная архитектура разбивает систему на отдельные независимые части.

Каждая система работает автономно и имеет свою зону ответственности



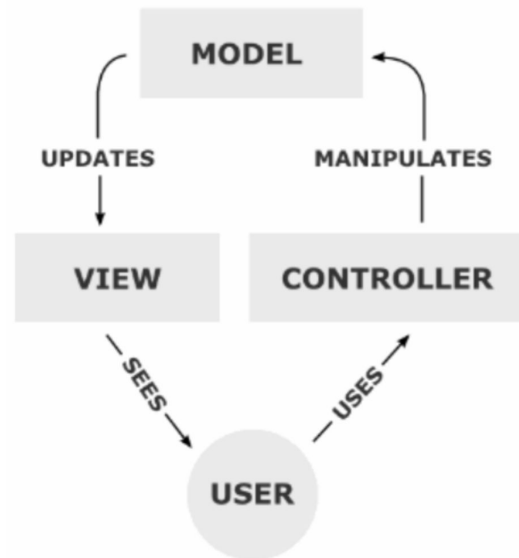
Model-View-Controller (MVC)

Основная цель применения этой концепции состоит в отделении бизнес-логики (модели) от её визуализации (представления, вида).

Модель предоставляет данные и реагирует на команды контроллера, изменяя своё состояние. Модель, за счёт независимости от визуального представления, может иметь несколько различных представлений для одной «модели»

Представление отвечает за отображение данных модели пользователю, реагируя на изменения модели.

Контроллер интерпретирует действия пользователя, оповещая модель о необходимости изменений. Контроллер обеспечивает «связь» между пользователем и системой. Контролирует и направляет данные от пользователя к системе и наоборот.



Maven

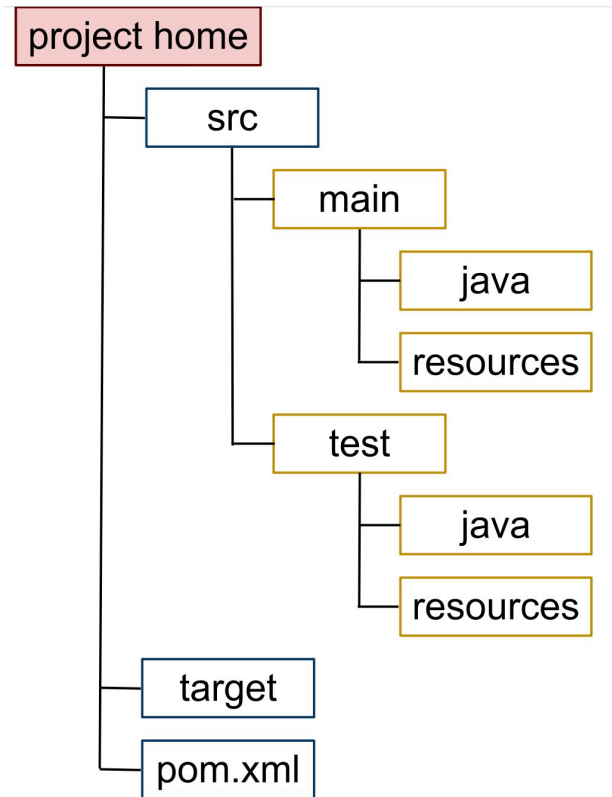
фреймворк для автоматизации сборки проектов на основе описания их структуры.

Помогает:

- Выстроить структуру проекта на основе архетипов
- Составить документацию по проекту
- Подключить зависимости в проект и управлять их версиями

Архетипы maven

Архетип — это шаблон для быстрого создания новых проектов с предустановленной структурой, файлами и конфигурациями.



pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  ..... xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  ..... xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
  ..... http://maven.apache.org/xsd/maven-4.0.0.xsd">
  .... <modelVersion>4.0.0</modelVersion>

  .... <groupId>com.example</groupId>
  .... <artifactId>my-app</artifactId>
  .... <version>1.0-SNAPSHOT</version>
</project>
```

Управление зависимостями

Осуществляется через блок <dependencies>

```
<dependencies>  
  <dependency>  
    <groupId>org.junit.jupiter</groupId>  
    <artifactId>junit-jupiter-engine</artifactId>  
    <version>5.9.1</version>  
    <scope>test</scope>  
  </dependency>  
</dependencies>
```

Репозитории Maven

- Локальный репозиторий (папка .m2)
- Maven Central
- Удаленный репозиторий (объявляется через `<repositories>`)

Scope

- **compile** - область по умолчанию, нужна для компиляции проекта.
- **provided** - область аналогична compile, за исключением того, что JDK или контейнер сам предоставит зависимость во время выполнения программы.
- **runtime** - зависимость нужна во время выполнения программы.
- **test** - зависимость не нужна для нормальной работы приложения, а нужна только для компиляции и запуска тестов.

Жизненный цикл сборки проекта

- **validate** - проверка правильности проекта и наличия всей необходимой информации.
- **compile** - компиляция исходного кода проекта
- **test** - тестирование скомпилированного исходного кода, используя подходящую среду модульного тестирования. Для проведения этих тестов не требуется упаковка или развертывание кода.
- **package** - упаковка скомпилированного кода в архив (jar, war).
- **verify** - проверка валидности артефакта: интеграционные тесты, процент покрытия кода тестами, анализ кода на уязвимости
- **install** - копирование пакет в локальный репозиторий, чтобы использовать его в качестве зависимости в других локальных проектах
- **deploy** - копирует итоговый пакет в удаленный репозиторий для обмена с другими разработчиками и проектами.
- **site** - создает HTML-сайт в **target/site** с Javadoc, отчётами и зависимостями.
- **clean** - удаляет все сгенерированные файлы, включая скомпилированный код

Плагины

Позволяют управлять жизненным циклом сборки

Плагин	Описание
maven-resources-plugin	Управляет Java-компиляцией
maven-resources-plugin	Включает ресурсы в сборку
maven-source-plugin	Включает исходный код в сборку
maven-dependency-plugin	Копирует библиотеки зависимостей
maven-jar-plugin	Создает итоговый jar-файл
maven-war-plugin	Создает итоговый war-файл
maven-surefire-plugin	Управляет запуском тестов
buildnumber-maven-plugin	Генерирует номер сборки

Логирование

процесс записи информации о происходящем в приложении.

- Упрощает поиск ошибок
- Помогает в мониторинге и аудите
- Сохраняет историю действий

Архитектура Logger

- Logger
- Appender
- Encoder

Уровни логирования

level of request p	effective level q					
	TRACE	DEBUG	INFO	WARN	ERROR	OFF
TRACE	YES	NO	NO	NO	NO	NO
DEBUG	YES	YES	NO	NO	NO	NO
INFO	YES	YES	YES	NO	NO	NO
WARN	YES	YES	YES	YES	NO	NO
ERROR	YES	YES	YES	YES	YES	NO

Тестирование

Модульное тестирование проверяет отдельные единицы кода (методы, классы) в изоляции от остальной системы.

JUnit 5 — стандартный фреймворк для модульного тестирования в Java, использующий аннотации для обозначения тестов и проверок.

Mockito — популярный фреймворк для создания моков (mock objects) в модульных тестах Java. Он позволяет имитировать поведение зависимостей, изолируя тестируемый класс от внешних сервисов, БД или API.

Spring Framework

Открытый фреймворк для создания приложений на языке Java.

Представляет собой набор независимых модулей для различных задач.

Берет на себя управление объектами, позволяя разработчику сосредоточиться на бизнес логике.

Модули Spring Framework

- **Spring Core** - основа (IoC контейнер).
- **Spring MVC** - для создания веб-приложений и REST API (контроллеры, обработка запросов).
- **Spring Data** - упрощает работу с базами данных (JPA, MongoDB, Redis).
- **Spring Security** - инструмент для аутентификации и авторизации (логин, пароль, роли, OAuth2).
- **Spring Boot** - надстройка над основным фреймворком для упрощенного создания и запуска приложений.
- **Spring Cloud** - Инструменты для создания микросервисов.

Основные концепции Spring Framework

- Dependency Injection (DI)
- Inversion of Control (IoC)
- Aspect Oriented Programming (AOP)

Dependency Injection

```
class Chef {  
    private Oven oven;  
  
    public Chef() {  
        this.oven = new ElectricOven();  
    }  
  
    public void cookPizza() {  
        oven.turnOn();  
        // ... Готовка  
    }  
}
```

Dependency Injection

```
class Chef {  
    private Oven oven;  
  
    // Внедрение через конструктор  
    public Chef(Oven oven) {  
        this.oven = oven;  
    }  
  
    public void cookPizza() {  
        oven.turnOn();  
        // ...  
    }  
}
```

Inversion of Control

```
Oven oven = new ElectricOven();  
Chef chef = new Chef(oven);
```

```
@Component  
public class UserRepository {}  
  
@Service  
public class UserService {  
    private final UserRepository repo;  
  
    @Autowired  
    public UserService(UserRepository repo) {  
        this.repo = repo;  
    }  
}
```

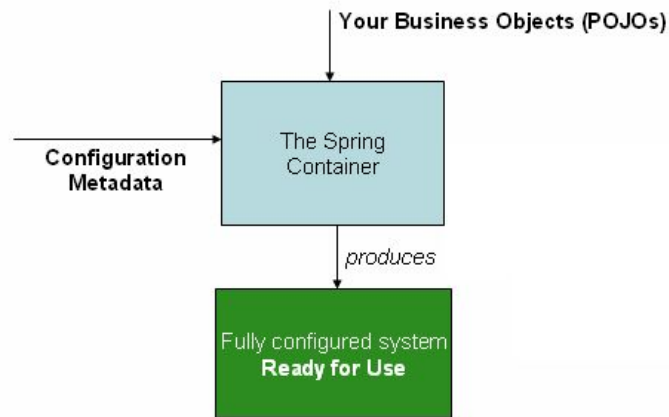
Spring Application Context

Интерфейс `org.springframework.context.ApplicationContext` представляет собой контейнер Spring IoC и отвечает за создание экземпляров, настройку и сборку компонентов.

Получает инструкции по созданию, настройке и сборке компонентов, считывая метаданные конфигурации.

Основные реализации:

- `AnnotationConfigApplicationContext`
- `ClassPathXmlApplicationContext`



Виды конфигурации Spring

- XML
- Annotation-based
- Java-based

XML-конфигурация

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
                           https://www.springframework.org/schema/beans/spring-beans.xsd">

    <!-- services -->

    <bean id="petStore" class="org.springframework.samples.jpetsy.store.services.PetStoreServiceImpl">
        <property name="accountDao" ref="accountDao"/>
        <property name="itemDao" ref="itemDao"/>
        <!-- additional collaborators and configuration for this bean go here -->
    </bean>

    <!-- more bean definitions for services go here -->

</beans>
```


XML-конфигурация

```
// create and configure beans
ApplicationContext context = new ClassPathXmlApplicationContext("services.xml", "daos.xml");

// retrieve configured instance
PetStoreService service = context.getBean("petStore", PetStoreService.class);

// use configured instance
List<String> userList = service.getUsernameList();
```

Annotation-based конфигурация

```
@Service
public class SimpleMovieLister {

    private final MovieFinder movieFinder;

    public SimpleMovieLister(MovieFinder movieFinder) {
        this.movieFinder = movieFinder;
    }
}
```

Java-based конфигурация

```
@Configuration
public class AppConfig {

    @Bean
    public MyServiceImpl myService() {
        return new MyServiceImpl();
    }
}
```

Bean

Объект, который создается, собирается и управляется контейнером Spring.

Чтобы объект стал бином, его нужно пометить одной из аннотаций:

- `@Component`
- `@Service`
- `@Repository`
- `@Controller`

Либо создать в конфигурации через аннотацию `@Bean`

Аннотация @ComponentScan

```
@Configuration
@ComponentScan(basePackages = "org.example")
public class AppConfig {
    // ...
}
```

```
@Configuration
@ComponentScan(basePackages = "org.example",
    includeFilters = @Filter(type = FilterType.REGEX, pattern = ".*Stub.*Repository"),
    excludeFilters = @Filter(Repository.class))
public class AppConfig {
    // ...
}
```

Scope

Scope	Описание
singleton	один бин для всего контейнера
prototype	один бин на каждый запрос из контейнера (каждую зависимость)
request	один бин для одного http-запроса
session	один бин на одну http-сессию
application	один бин на один ServletContext

Жизненный цикл бина (lifecycle)

- Создание
- Внедрение зависимостей
- Настройка (**@PostConstruct**)
- Использование
- Уничтожение (**@PreDestroy**)

```
public class CachingMovieLister {  
  
    @PostConstruct  
    public void populateMovieCache() {  
        // populates the movie cache upon initialization...  
    }  
  
    @PreDestroy  
    public void clearMovieCache() {  
        // clears the movie cache upon destruction...  
    }  
}
```

Внедрение зависимостей через конструктор

```
public class MovieRecommender {  
  
    private final CustomerPreferenceDao customerPreferenceDao;  
  
    @Autowired  
    public MovieRecommender(CustomerPreferenceDao customerPreferenceDao) {  
        this.customerPreferenceDao = customerPreferenceDao;  
    }  
  
    // ...  
}
```


Внедрение зависимостей через setter

```
public class SimpleMovieLister {  
  
    private MovieFinder movieFinder;  
  
    @Autowired  
    public void setMovieFinder(MovieFinder movieFinder) {  
        this.movieFinder = movieFinder;  
    }  
  
    // ...  
}
```

Внедрение списков и мап

```
public class MovieRecommender {  
  
    @Autowired  
    private MovieCatalog[] movieCatalogs;  
  
    // ...  
}
```

```
public class MovieRecommender {  
  
    private Map<String, MovieCatalog> movieCatalogs;  
  
    @Autowired  
    public void setMovieCatalogs(Map<String, MovieCatalog> movieCatalogs) {  
        this.movieCatalogs = movieCatalogs;  
    }  
  
    // ...  
}
```

Опциональное внедрение зависимостей

```
public class SimpleMovieLister {  
  
    private MovieFinder movieFinder;  
  
    @Autowired(required = false)  
    public void setMovieFinder(MovieFinder movieFinder) {  
        this.movieFinder = movieFinder;  
    }  
  
    // ...  
}
```

Опциональное внедрение зависимостей

```
public class SimpleMovieLister {  
  
    @Autowired  
    public void setMovieFinder(@Nullable MovieFinder movieFinder) {  
        ...  
    }  
}
```

Внедрение зависимости через Java-конфигурацию

```
@Configuration
public class AppConfig {

    @Bean
    public TransferService transferService(AccountRepository accountRepository) {
        return new TransferServiceImpl(accountRepository);
    }
}
```

Конфликты при внедрении зависимостей

Если в контексте существует более одного бина одинакового типа, то Spring не будет пытаться угадать какой из них внедрять, а сразу выдаст ошибку.

Есть два решения:

- @Primary
- @Qualifier

@Primary

```
@Configuration
public class MovieConfiguration {

    @Bean
    @Primary
    public MovieCatalog firstMovieCatalog() { ... }

    @Bean
    public MovieCatalog secondMovieCatalog() { ... }

    // ...
}
```

@Qualifier

```
public class MovieRecommender {  
  
    private final MovieCatalog movieCatalog;  
  
    private final CustomerPreferenceDao customerPreferenceDao;  
  
    @Autowired  
    public void prepare(@Qualifier("main") MovieCatalog movieCatalog,  
                        CustomerPreferenceDao customerPreferenceDao) {  
        this.movieCatalog = movieCatalog;  
        this.customerPreferenceDao = customerPreferenceDao;  
    }  
  
    // ...  
}
```


Внедрение констант

```
@Component
public class MovieRecommender {

    private final String catalog;

    public MovieRecommender(@Value("${catalog.name}") String catalog) {
        this.catalog = catalog;
    }
}
```

Значения по умолчанию

```
@Component
public class MovieRecommender {

    private final String catalog;

    public MovieRecommender(@Value("${catalog.name:defaultCatalog}") String catalog) {
        this.catalog = catalog;
    }
}
```

Aspect-oriented Programming (AOP)

подход к программированию, который добавляет ещё один способ разбиения программы на части поверх ООП, чтобы удобно выносить сквозные задачи вроде логирования, транзакций, безопасности и т. д.

Пример

```
@Service
public class StudentService {

    public void registerStudent(String name) { no usages
        // бизнес-логика
        System.out.println("Регистрация ученика: " + name);
    }

    public String getStudentInfo(long id) { no usages
        // Получить данные ученика
        return "Информация об ученике с id = " + id;
    }
}
```

Пример

```
@Aspect
@Component
public class LoggingAspect {

    @Around("execution(* com.example.school.StudentService.*(..))")
    public Object logAround(ProceedingJoinPoint joinPoint) throws Throwable {
        long start = System.currentTimeMillis();

        String methodName = joinPoint.getSignature().getName();
        Object[] args = joinPoint.getArgs();
        System.out.println("Метод " + methodName + " вызван с аргументами " + Arrays.toString(args));

        // Выполнение самого метода
        Object result = joinPoint.proceed();

        long time = System.currentTimeMillis() - start;
        System.out.println("Метод " + methodName + " завершился за " + time + " мс, результат: " + result);

        return result;
    }
}
```

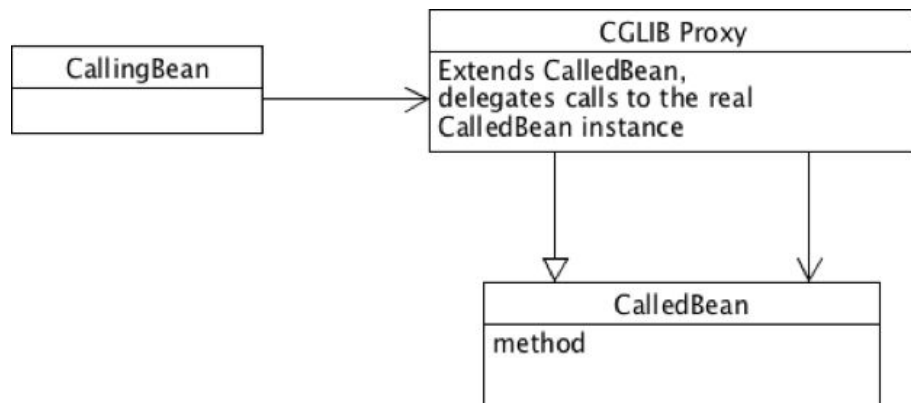
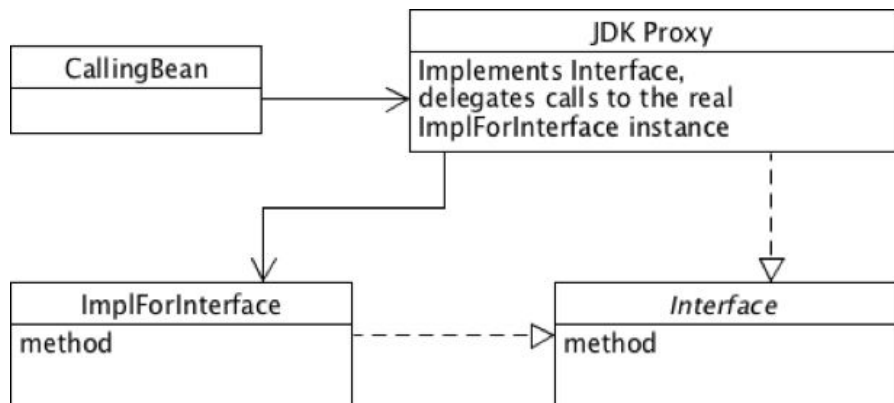
Подключение

```
@Configuration  
@EnableAspectJAutoProxy  
public class ApplicationConfiguration {  
}
```

Proxy

JDK-прокси	CGLIB
создаётся через <code>java.lang.reflect.Proxy</code> , прокси реализует один или несколько интерфейсов целевого бина, сам класс бина не наследуется	создаётся путём генерации наследника целевого класса (через bytecode generation), proxied-класс расширяется, а методы переопределяются
перехватываются только вызовы публичных методов интерфейса, вызываемых через ссылку на прокси	перехватываются <code>public</code> и <code>protected</code> методы, а при необходимости и <code>package-private</code> , т.к. прокси — это подкласс
требует, чтобы бин реализовывал хотя бы один интерфейс	работает и без интерфейсов, но не может проксировать <code>final</code> -классы и <code>final</code> -методы (их нельзя переопределить)

JDK vs CGLIB



Объявление аспектов

```
@Aspect
public class NotVeryUsefulAspect {
}
```

```
public class ApplicationConfiguration {

    @Bean
    public NotVeryUsefulAspect myAspect() {
        NotVeryUsefulAspect myAspect = new NotVeryUsefulAspect();
        // Configure properties of the aspect here
        return myAspect;
    }
}
```

Объявление Pointcut

```
public class Pointcuts {  
  
    @Pointcut("execution(public * *(..))")  
    public void publicMethod() {} ❶  
  
    @Pointcut("within(com.xyz.trading..*)")  
    public void inTrading() {} ❷  
  
    @Pointcut("publicMethod() && inTrading()")  
    public void tradingOperation() {} ❸  
}
```

Указатели срезов (Pointcut Designators)

Указатель	За что отвечает
execution	сопоставление с выполнением метода
within	ограничивает join point'ы методами, объявленными в указанных типах или пакетах
this	join point'ы, где ссылка на бин (прокси Spring AOP) является экземпляром указанного типа
target	join point'ы, где целевой объект (реальный бин за прокси) является экземпляром указанного типа
args	join point'ы, где аргументы во время выполнения являются экземплярами указанных типов
@annotation	методы, имеющие указанную аннотацию.

Объявление советов (Before Advice)

```
@Aspect
public class BeforeExample {

    @Before("com.xyz.CommonPointcuts.dataAccessOperation()")
    public void doAccessCheck() {
        // ...
    }
}
```

```
@Aspect
public class BeforeExample {

    @Before("execution(* com.xyz.dao.*.*(..))")
    public void doAccessCheck() {
        // ...
    }
}
```

Объявление советов (After Advice)

```
@Aspect
public class AfterReturningExample {

    @AfterReturning("execution(* com.xyz.dao.*.*(..))")
    public void doAccessCheck() {
        // ...
    }
}

@Aspect
public class AfterReturningExample {

    @AfterReturning(
        pointcut="execution(* com.xyz.dao.*.*(..))",
        returning="retVal")
    public void doAccessCheck(Object retVal) {
        // ...
    }
}
```

Объявление советов (After Throwing Advice)

```
@Aspect
public class AfterThrowingExample {

    @AfterThrowing("execution(* com.xyz.dao.*.*(..))")
    public void doRecoveryActions() {
        // ...
    }
}
```

```
@Aspect
public class AfterThrowingExample {

    @AfterThrowing(
        pointcut="execution(* com.xyz.dao.*.*(..))",
        throwing="ex")
    public void doRecoveryActions(DataAccessException ex) {
        // ...
    }
}
```

Объявление советов (After Advice)

```
@Aspect
public class AfterFinallyExample {

    @After("execution(* com.xyz.dao.*.*(..))")
    public void doReleaseLock() {
        // ...
    }
}
```

Объявление советов (Around Advice)

```
@Aspect
public class AroundExample {

    @Around("execution(* com.xyz..service.*(..))")
    public Object doBasicProfiling(ProceedingJoinPoint pjp) throws Throwable {
        // start stopwatch
        Object retVal = pjp.proceed();
        // stop stopwatch
        return retVal;
    }
}
```


Методы JoinPoint

Object `getThis()` - возвращает прокси объект

Object `getTarget()` - возвращает целевой объект

Object[] `getArgs()` - возвращает аргументы метода

Signature `getSignature()` - возвращает описание метода

Пример обработки метода с аннотацией

```
@Before("com.xyz.Pointcuts.publicMethod() && @annotation(auditable)") ❶  
public void audit(Auditable auditable) {  
    AuditCode code = auditable.value();  
    // ...  
}
```

Spring DAO

Упрощает работу с технологиями доступа к данным, позволяет легко переключаться между различными технологиями.

Преобразует специфичные исключения типа `SQLException` в собственную иерархию исключений с корнем `DataAccessException`.

Пример из прошлого семестра

```
Connection conn = null;
try {
    String url = "jdbc:postgresql://localhost:5432/postgres";
    conn = DriverManager.getConnection(url, user: "user", password: "pass");
    String query = "select * from newtable";
    try (Statement stmt = conn.createStatement()) {
        var i : int = stmt.executeUpdate( sql: "insert into newtable(name) values ('kniit')");
        System.out.println("rows updated " + i);

        ResultSet rs = stmt.executeQuery(query);
        while (rs.next()) {
            int id = rs.getInt( columnLabel: "id");
            String name = rs.getString( columnLabel: "name");
            System.out.println(id + " " + name);
        }
    } catch (SQLException e ) {
        throw new Error( message: "Problem", e);
    }
} catch (SQLException e) {
    throw new Error( message: "Problem", e);
} finally {
    try {
        if (conn != null) {
            conn.close();
        }
    } catch (SQLException ex) {
        System.out.println(ex.getMessage());
    }
}
```

Spring JdbcTemplate

JdbcTemplate - основной способ взаимодействия с базой, позволяет выполнять запросы, возвращает результаты.

NamedParameterJdbcTemplate - декоратор над **JdbcTemplate**, позволяет заменить "?" в подстановках на имя параметра.

Пример JdbcTemplate Query

```
int countOfActorsNamedJoe = this.jdbcTemplate.queryForObject(
    "select count(*) from t_actor where first_name = ?", Integer.class, "Joe");

Actor actor = jdbcTemplate.queryForObject(
    "select first_name, last_name from t_actor where id = ?",
    (resultSet, rowNum) -> {
        Actor newActor = new Actor();
        newActor.setFirstName(resultSet.getString("first_name"));
        newActor.setLastName(resultSet.getString("last_name"));
        return newActor;
    },

List<Actor> actors = this.jdbcTemplate.query(
    "select first_name, last_name from t_actor",
    (resultSet, rowNum) -> {
        Actor actor = new Actor();
        actor.setFirstName(resultSet.getString("first_name"));
        actor.setLastName(resultSet.getString("last_name"));
        return actor;
    });
```

Insert, Update, Delete

```
this.jdbcTemplate.update(  
    "insert into t_actor (first_name, last_name) values (?, ?)",  
    "Leonor", "Watling");
```

```
this.jdbcTemplate.update(  
    "update t_actor set last_name = ? where id = ?",  
    "Banjo", 5276L);
```

```
this.jdbcTemplate.update(  
    "delete from t_actor where id = ?",  
    Long.valueOf(actorId));
```

Создание JdbcTemplate

```
@Repository
public class JdbcCorporateEventRepository implements CorporateEventRepository {

    private JdbcTemplate jdbcTemplate;

    // Implicitly autowire the DataSource constructor parameter
    public JdbcCorporateEventRepository(DataSource dataSource) {
        this.jdbcTemplate = new JdbcTemplate(dataSource);
    }

    // JDBC-backed implementations of the methods on the CorporateEventRepository follow...
}
```


NamedParameterJdbcTemplate

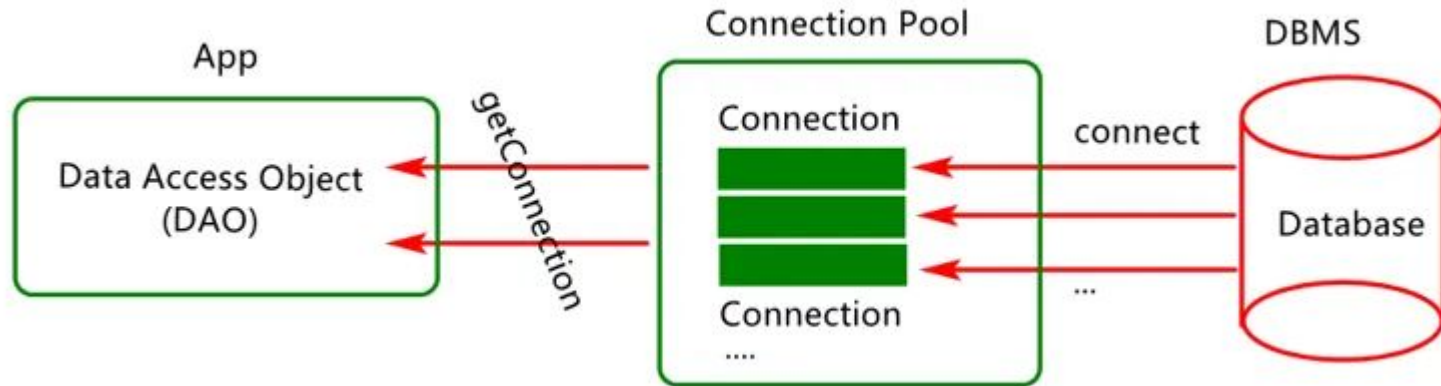
```
private NamedParameterJdbcTemplate namedParameterJdbcTemplate;  
  
public void setDataSource(DataSource dataSource) {  
    this.namedParameterJdbcTemplate = new NamedParameterJdbcTemplate(dataSource);  
}  
  
public int countOfActorsByFirstName(String firstName) {  
    String sql = "select count(*) from t_actor where first_name = :first_name";  
    SqlParameterSource namedParameters = new MapSqlParameterSource("first_name", firstName);  
    return this.namedParameterJdbcTemplate.queryForObject(sql, namedParameters, Integer.class);  
}
```

DataSource

Предоставляет средства управления соединением с БД.

```
@Bean
DataSource dataSource() {
    DriverManagerDataSource dataSource = new DriverManagerDataSource();
    dataSource.setDriverClassName("org.hsqldb.jdbcDriver");
    dataSource.setUrl("jdbc:hsqldb:hsqldb://localhost:");
    dataSource.setUsername("sa");
    dataSource.setPassword("");
    return dataSource;
}
```

Connection Pool



HikariCP

```
@Configuration
public class DataSourceConfig {
    @Bean
    @ConfigurationProperties("spring.datasource.hikari")
    public HikariDataSource dataSource() {
        return DataSourceBuilder.create()
            .type(HikariDataSource.class)
            .build();
    }
}

spring.datasource.url=jdbc:postgresql://localhost:5432/mydb
spring.datasource.username=user
spring.datasource.password=pass
spring.datasource.hikari.maximum-pool-size=20
spring.datasource.hikari.minimum-idle=5
spring.datasource.hikari.connection-timeout=30000
spring.datasource.hikari.idle-timeout=600000
spring.datasource.hikari.max-lifetime=1800000
```

Maven BOM

```
<project ...>

  <modelVersion>4.0.0</modelVersion>
  <groupId>example</groupId>
  <artifactId>Example-BOM</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <packaging>pom</packaging>
  <name>Example-BOM</name>
  <description>parent pom</description>

  <dependencyManagement>
    <dependencies>
      <dependency>
        <groupId>proj</groupId>
        <artifactId>a</artifactId>
        <version>1.2</version>
      </dependency>

      <dependency>
        <groupId>proj</groupId>
        <artifactId>b</artifactId>
        <version>1.0</version>
        <scope>compile</scope>
      </dependency>

      <dependency>
        <groupId>proj</groupId>
        <artifactId>c</artifactId>
        <version>1.0</version>
        <scope>compile</scope>
      </dependency>
    </dependencies>
  </dependencyManagement>
</project>
```

Использование

Наследование

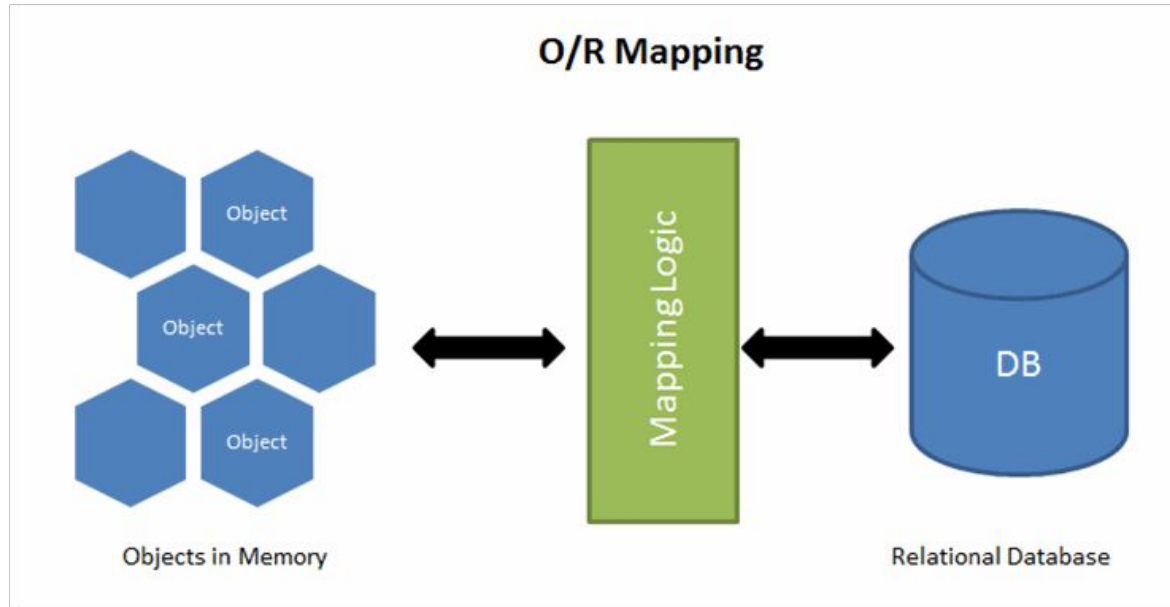
```
<project ...>
  <modelVersion>4.0.0</modelVersion>
  <groupId>example</groupId>
  <artifactId>Test</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <packaging>pom</packaging>
  <name>Test</name>
  <parent>
    <groupId>example</groupId>
    <artifactId>Example-BOM</artifactId>
    <version>0.0.1-SNAPSHOT</version>
  </parent>
</project>
```

Через <dependencyManagement>

```
<project ...>
  <modelVersion>4.0.0</modelVersion>
  <groupId>baeldung</groupId>
  <artifactId>Test</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <packaging>pom</packaging>
  <name>Test</name>

  <dependencyManagement>
    <dependencies>
      <dependency>
        <groupId>baeldung</groupId>
        <artifactId>BaeIdung-BOM</artifactId>
        <version>0.0.1-SNAPSHOT</version>
        <type>pom</type>
        <scope>import</scope>
      </dependency>
    </dependencies>
  </dependencyManagement>
</project>
```

Object-Relational Mapping (ORM)



Описание сущности

```
@Entity
@Table(name = "users")
public class User {

    @Id no usages
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String username;

    @Column(nullable = false)
    private String email;

    @OneToMany(
        mappedBy = "user"
    )
    private Set<UserRole> userRoles = new HashSet<>();

    @ManyToMany(fetch = FetchType.LAZY)
    @JoinTable(
        name = "users_groups",
        joinColumns = @JoinColumn(name = "user_id"),
        inverseJoinColumns = @JoinColumn(name = "group_id")
    )
    private Set<Group> groups = new HashSet<>();
}
```

```
@Entity
@Table(name = "user_roles")
public class UserRole {

    @Id no usages
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "user_id", nullable = false)
    private User user;

    @Column(name = "role_name", nullable = false)
    private String roleName;
}
```

```
@Entity
@Table(name = "groups")
public class Group {

    @Id no usages
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String name;

    @ManyToMany(mappedBy = "groups")
    private Set<User> users = new HashSet<>();
}
```


Spring Data

Миссия **Spring Data** — предоставить привычную и согласованную программную модель доступа к данным на основе Spring, сохраняя при этом особые характеристики базового хранилища данных.

Он упрощает использование технологий доступа к данным, реляционных и нереляционных баз данных.

Это проект, включающий множество подпроектов, специфичных для конкретной базы данных.

Проекты разрабатываются в сотрудничестве со многими компаниями и разработчиками, стоящими за различными технологиями.

Spring Data

Основная ответственность:

1. Создание экземпляра с использованием одного из доступных конструкторов.
2. Заполнение экземпляра для материализации всех доступных свойств.

Алгоритм создания объектов

1. Если есть статический фабричный метод с аннотацией `@PersistenceCreator`, используется он.
2. Если есть единственный конструктор, используется он.
3. Если конструкторов несколько и ровно один аннотирован `@PersistenceCreator`, используется он.
4. Если тип — Java-Record, используется канонический конструктор.
5. Если есть конструктор без аргументов, используется он; остальные конструкторы игнорируются.

Repository

```
public interface CrudRepository<T, ID> extends Repository<T, ID> {  
  
    <S extends T> S save(S entity);           ❶  
  
    Optional<T> findById(ID primaryKey);      ❷  
  
    Iterable<T> findAll();                     ❸  
  
    long count();                              ❹  
  
    void delete(T entity);                     ❺  
  
    boolean existsById(ID primaryKey);         ❻  
  
    // ... more functionality omitted.  
}
```

PagingAndSortingRepository

```
public interface PagingAndSortingRepository<T, ID> {  
  
    Iterable<T> findAll(Sort sort);  
  
    Page<T> findAll(Pageable pageable);  
}
```

```
PagingAndSortingRepository<User, Long> repository = // ... get access to a bean  
Page<User> users = repository.findAll(PageRequest.of(1, 20));
```

Создание и использование репозитория

```
interface PersonRepository extends Repository<Person, Long> {  
    List<Person> findByLastname(String lastname);  
}
```

```
@EnableJpaRepositories  
class Config { ... }
```

```
class SomeClient {  
  
    private final PersonRepository repository;  
  
    SomeClient(PersonRepository repository) {  
        this.repository = repository;  
    }  
  
    void doSomething() {  
        List<Person> persons = repository.findByLastname("Matthews");  
    }  
}
```

@NoRepositoryBean

```
@NoRepositoryBean
interface MyBaseRepository<T, ID> extends Repository<T, ID> {

    Optional<T> findById(ID id);

    <S extends T> S save(S entity);
}

interface UserRepository extends MyBaseRepository<User, Long> {
    User findByEmailAddress(EmailAddress emailAddress);
}
```

Методы для запросов

```
interface PersonRepository extends Repository<Person, Long> {  
  
    List<Person> findByEmailAddressAndLastname(EmailAddress emailAddress, String lastname);  
  
    // Enables the distinct flag for the query  
    List<Person> findDistinctPeopleByLastnameOrFirstname(String lastname, String firstname);  
    List<Person> findPeopleDistinctByLastnameOrFirstname(String lastname, String firstname);  
  
    // Enabling ignoring case for an individual property  
    List<Person> findByLastnameIgnoreCase(String lastname);  
    // Enabling ignoring case for all suitable properties  
    List<Person> findByLastnameAndFirstnameAllIgnoreCase(String lastname, String firstname);  
  
    // Enabling static ORDER BY for a query  
    List<Person> findByLastnameOrderByFirstnameAsc(String lastname);  
    List<Person> findByLastnameOrderByFirstnameDesc(String lastname);  
}
```


Разрешение конфликтов

```
List<Person> findByAddressZipCode(ZipCode zipCode);
```

```
List<Person> findByAddress_ZipCode(ZipCode zipCode);
```

@Query

```
public interface UserRepository extends JpaRepository<User, Long> {

    @Query("SELECT u FROM User u WHERE u.username = ?1")
    Optional<User> findByUsername(String username);

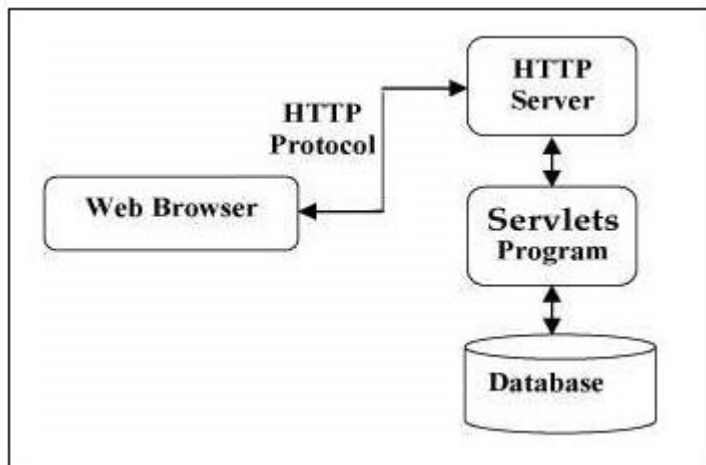
    @Query("SELECT u FROM User u WHERE u.lastname = :lastname")
    Optional<User> findByLastname(String lastname);

    @Query("SELECT u FROM User u WHERE u.age > :minAge AND u.department = :dept")
    List<User> findByAgeGreaterThanAndDepartment(
        @Param("minAge") int minAge,
        @Param("dept") String dept
    );
}
```

Native Query

```
public interface UserRepository extends JpaRepository<User, Long> {  
  
    @Query(  
        value = "SELECT u.* FROM users u WHERE u.email LIKE %:email%",  
        nativeQuery = true  
    )  
    List<User> findByEmailContaining(@Param("email") String email);  
}
```

Путь запроса



http-запрос

Стартовая строка

Заголовки

Пустая строка

Тело

```
GET /search?q=java HTTP/1.1
```

```
Host: yandex.ru
```

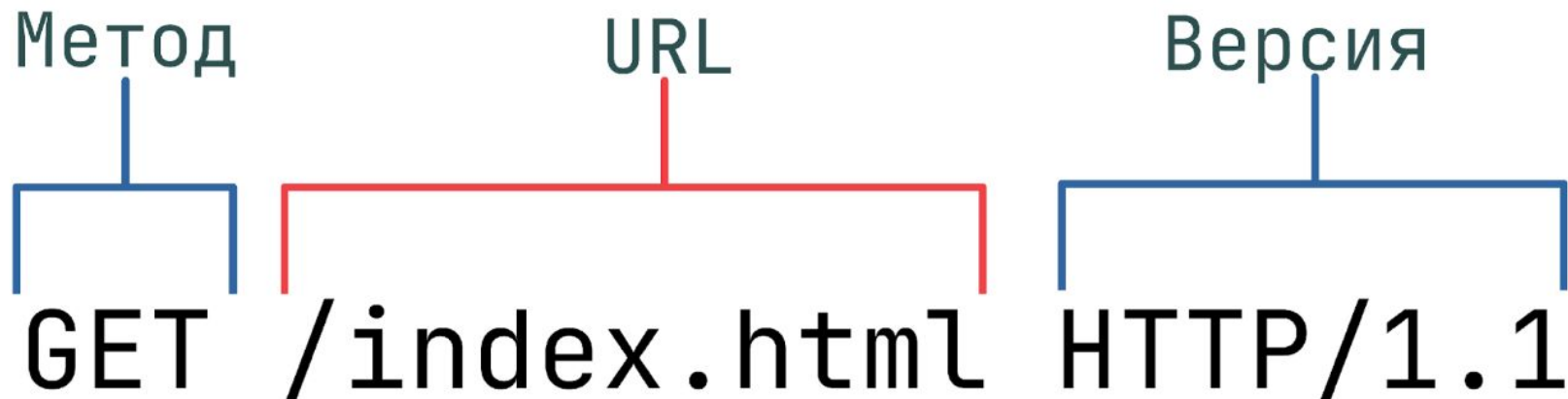
```
User-Agent: Chrome/120.0
```

```
Accept: text/html
```

```
Accept-Language: ru-RU
```

```
Connection: keep-alive
```

Стартовая строка



Методы

МЕТОД	ОПИСАНИЕ	ОСОБЕННОСТИ
GET	Используется для запроса некоторых данных с сервера.	Позволяет получить, но не изменять данные. Запрос остаётся в истории браузера и может быть сохранён в закладках. Чаще всего применяется для получения контента с сервера.
POST	Используется для отправки данных на сервер для создания или обновления запрашиваемого ресурса. Данные для этого указываются в теле запроса.	Запрос не сохраняется в истории браузера и не может быть сохранён в закладки. В ответ на него сервер выполняет фоновые процессы по обновлению и отчитывается о результате. В ответ он может прислать например адрес новой созданной записи.
PUT	Также используется для отправки данных на сервер для создания или обновления запрашиваемого ресурса.	В отличие от POST не требует от сервера ответа и выполнения фоновых процессов. Например, с его помощью можно изменить уже существующую запись.
PATCH	Используется для частичного изменения ресурса.	Включает в себя не полностью новое содержимое, а инструкцию по обновлению, чем отличается от метода PUT.
DELETE	Используется для удаления запрашиваемого ресурса.	Полностью удаляет ненужную информацию.

Заголовки

Host	Используется для указания того, с какого конкретно хоста запрашивается ресурс. В качестве возможных значений могут использоваться как доменные имена, так и IP-адреса.
User-Agent	Заголовок используется для описания клиента, который запрашивает ресурс.
Accept	Позволяет указать, какой тип медиафайлов принимает клиент.
Cookie	Данный заголовок может содержать в себе одну или несколько пар «Куки-Значение» в формате <code>cookie=value</code> .
Authorization	Используется в качестве еще одного метода идентификации клиента на сервере. После успешной идентификации сервер возвращает токен, уникальный для каждого конкретного клиента.

Тело запроса

```
POST /api/users HTTP/1.1
Host: example.com
Content-Type: application/json
Content-Length: 45
User-Agent: Chrome/120
```

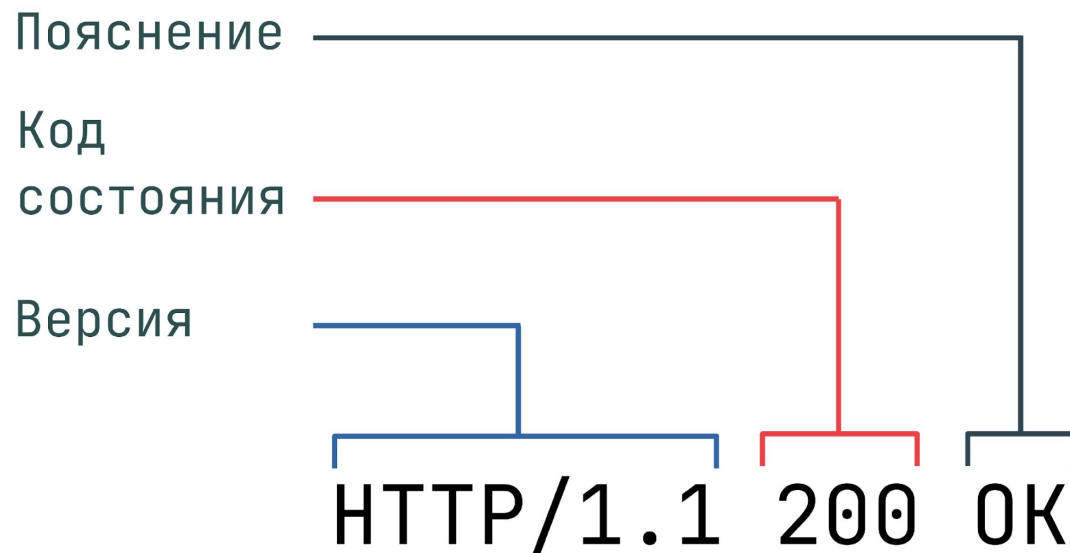
```
{
  "name": "Иван Иванов",
  "email": "ivan@example.com",
  "age": 25
}
```

http-ответ

```
HTTP/1.1 201 Created  
Content-Type: application/json
```

```
{  
  "id": 123,  
  "name": "Иван Иванов",  
  "email": "ivan@example.com"  
}
```

Строка статуса



Группы кодов ответа

HTTP Status Codes



Коды состояния

200 OK	Возвращается в случае успешной обработки запроса, при этом тело ответа обычно содержит запрошенный ресурс.
302 Found	Перенаправляет клиента на другой URL. Например, данный код может прийти, если клиент успешно прошел процедуру аутентификации и теперь может перейти на страницу своей учетной записи.
400 Bad Request	Данный код можно увидеть, если запрос был сформирован с ошибками. Например, в нем отсутствовали символы завершения строки.
403 Forbidden	Означает, что клиент не обладает достаточными правами доступа к запрошенному ресурсу.
404 Not Found	Данный код можно увидеть, если запросить у сервера ресурс, которого не существует на сервере.
500 Internal Error	Данный код возвращается сервером, когда он не может по определенным причинам обработать запрос.

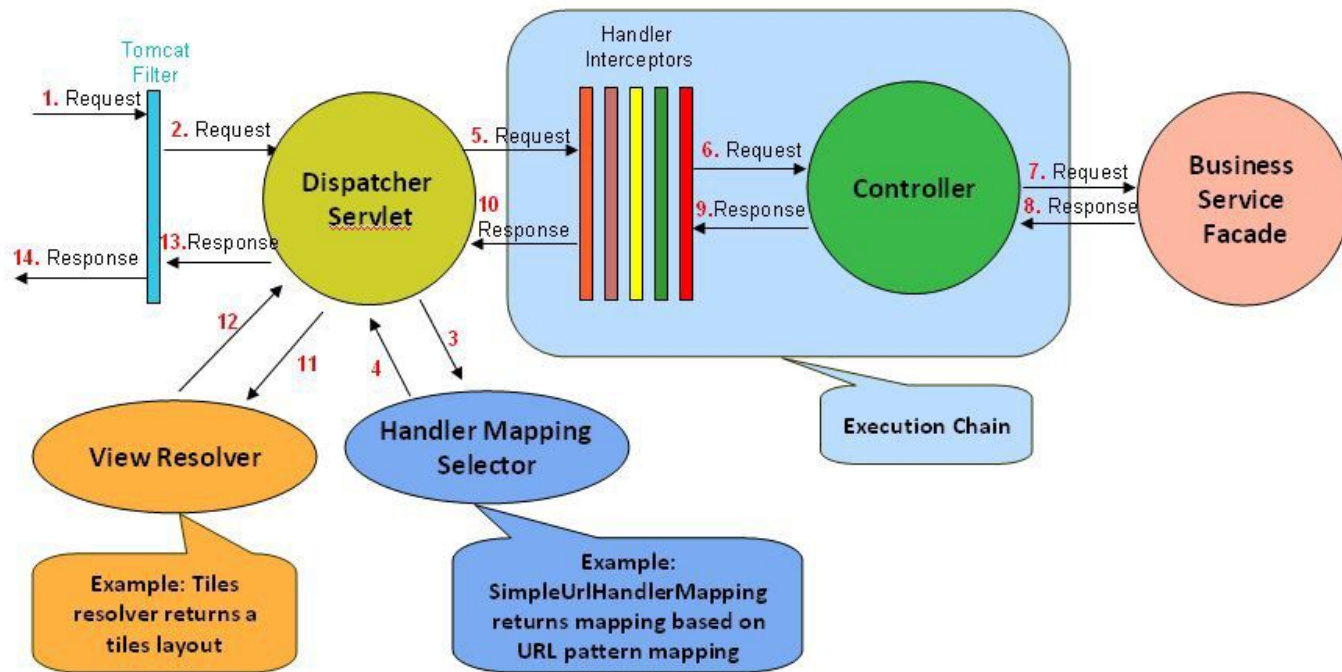
http-сервлет

Сервлет — это специальный интерфейс в языке программирования Java, который нужен для обмена информацией между серверной и клиентской частью программы.

Сервлет размещается на сервере, принимает запросы от клиента и отправляет на них ответы.

Он управляет общением между разными частями приложения.

Spring MVC



@EnableWebMvc

```
@Configuration  
@EnableWebMvc  
public class WebConfiguration {  
}
```


Controller

```
@Controller
public class HelloController {

    @GetMapping("/hello")
    public String handle(Model model) {
        model.addAttribute("message", "Hello World!");
        return "index";
    }
}
```

RestController

```
@RestController
@RequestMapping("/persons")
class PersonController {

    @GetMapping("/{id}")
    public Person getPerson(@PathVariable Long id) {
        // ...
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public void add(@RequestBody Person person) {
        // ...
    }
}
```

@RequestHeader

Host	localhost:8080
Accept	text/html,application/xhtml+xml,application/xml;q=0.9
Accept-Language	fr,en-gb;q=0.7,en;q=0.3
Accept-Encoding	gzip,deflate
Accept-Charset	ISO-8859-1,utf-8;q=0.7,*;q=0.7
Keep-Alive	300

```
@GetMapping("/demo")
public void handle(
    @RequestHeader("Accept-Encoding") String encoding, ❶
    @RequestHeader("Keep-Alive") long keepAlive) { ❷
    //...
}
```

ResponseEntity

```
@GetMapping("/something")
public ResponseEntity<String> handle() {
    String body = ... ;
    String etag = ... ;
    return ResponseEntity.ok().eTag(etag).body(body);
}
```

ControllerAdvice и ExceptionHandler

```
@ExceptionHandler({FileSystemException.class, RemoteException.class})  
public ResponseEntity<String> handleIOException(IOException ex) {  
    return ResponseEntity.internalServerError().body(ex.getMessage());  
}
```

Spring Boot

Фреймворк, предназначенный для упрощения и ускорения разработки автономных, готовых к промышленной эксплуатации приложений.

Ключевые особенности:

- Автоконфигурация
- Минимальная настройка
- Стартеры

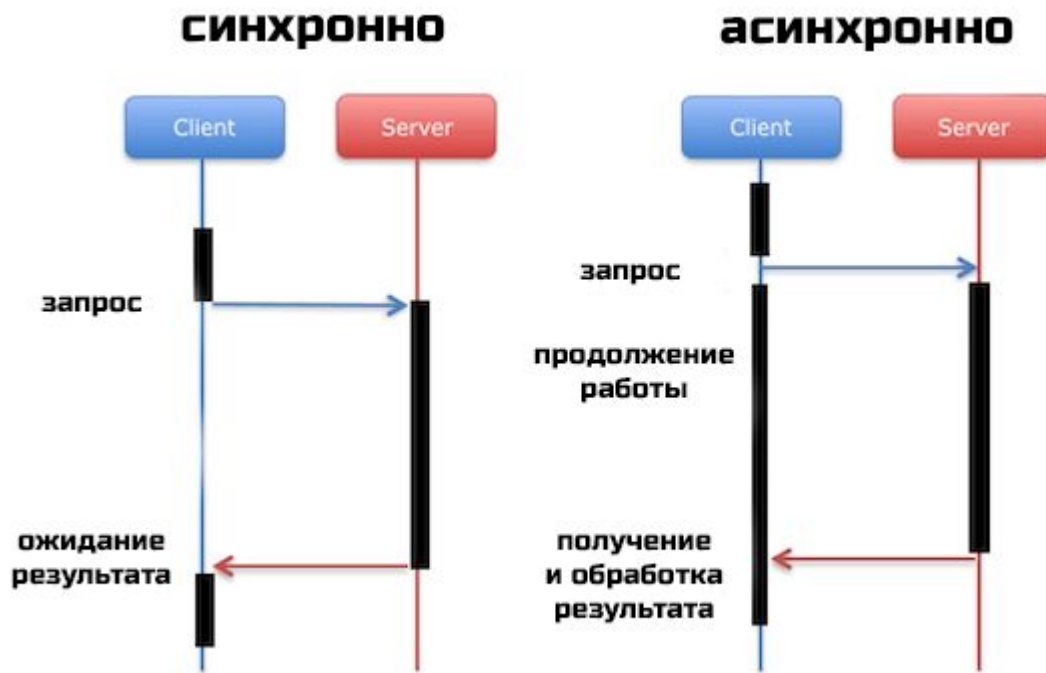
@SpringBootApplication

Компонент	Что делает	Пример эффекта
@Configuration	Класс как источник бинов (Spring IoC)	main() создаёт ApplicationContext
@EnableAutoConfiguration	Автонастройка по classpath	spring-boot-starter-web → embedded Tomcat + Jackson + MVC
@ComponentScan	Поиск @Component в пакете и подпакетах	@RestController в com.example.controller → авто-регистрация

@ConditionalOn...

- `@ConditionalOnClass` — сработает, если в classpath есть указанный класс.
- `@ConditionalOnMissingClass` — сработает, если класса нет.
- `@ConditionalOnBean` — сработает, если в контексте уже есть нужный бин.
- `@ConditionalOnMissingBean` — сработает, если нужного бина ещё нет.
- `@ConditionalOnProperty` — сработает, если в конфигурации задано определённое СВОЙСТВО.
- `@ConditionalOnWebApplication` — сработает только в веб-приложении.
- `@ConditionalOnResource` — сработает, если есть нужный ресурс.

Синхронное vs Асинхронное взаимодействие



Модели push и pull

Push: система сама отправляет сообщение потребителю.

Pull: потребитель сам запрашивает новые сообщения, когда готов их читать.

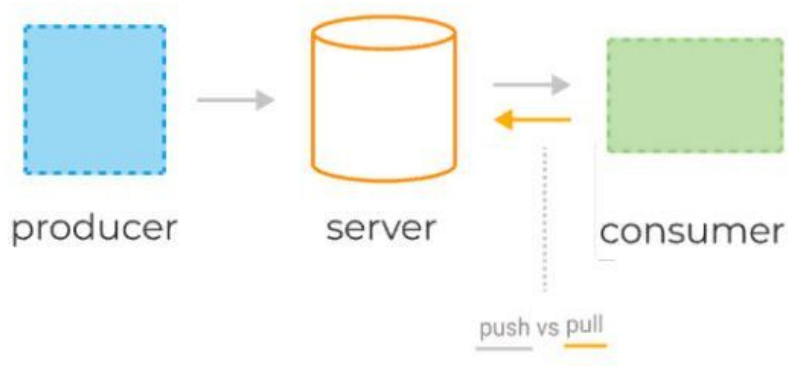
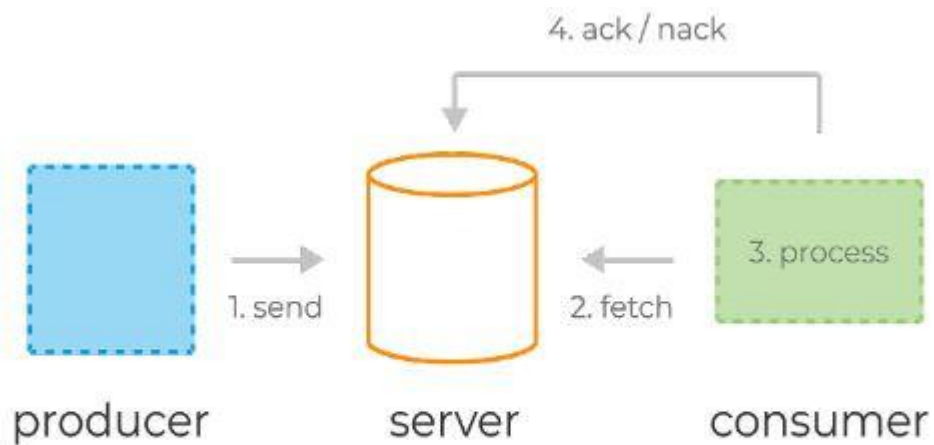
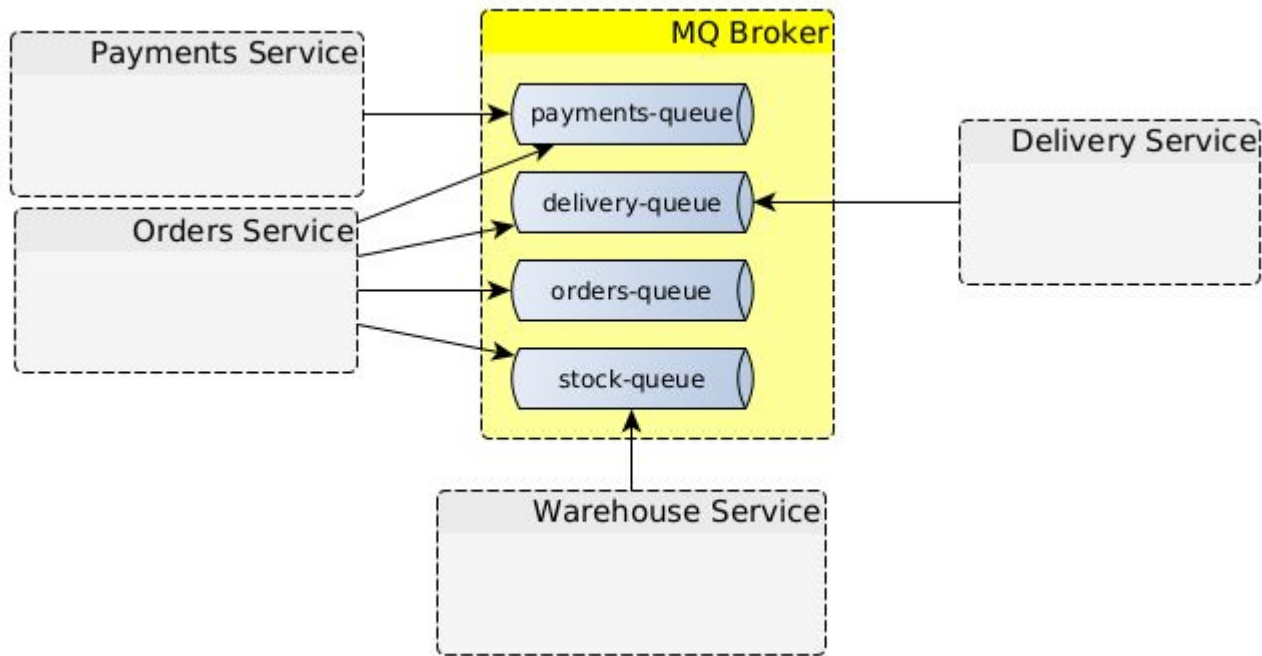


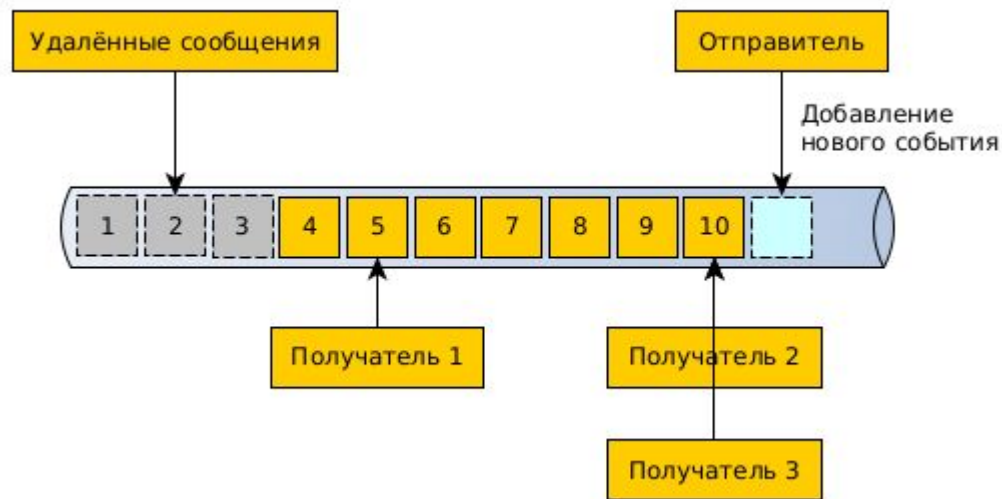
Схема взаимодействия



Очереди



Стриминговые системы



Apache Kafka

Broker — узел кластера, который хранит данные.

Topic разбит на **partitions**, чтобы масштабировать запись и чтение.

Producer отправляет сообщение в **topic**.

Consumer group позволяет нескольким потребителям работать параллельно, не дублируя обработку одной и той же **partition** внутри группы.

Apache Kafka

